

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Resources

Pods

Welcome to this comprehensive guide on Kubernetes pods. In this article, we will explore the essentials of pods, their role in container orchestration, and how they simplify scaling and deployment. Before proceeding, ensure you have:

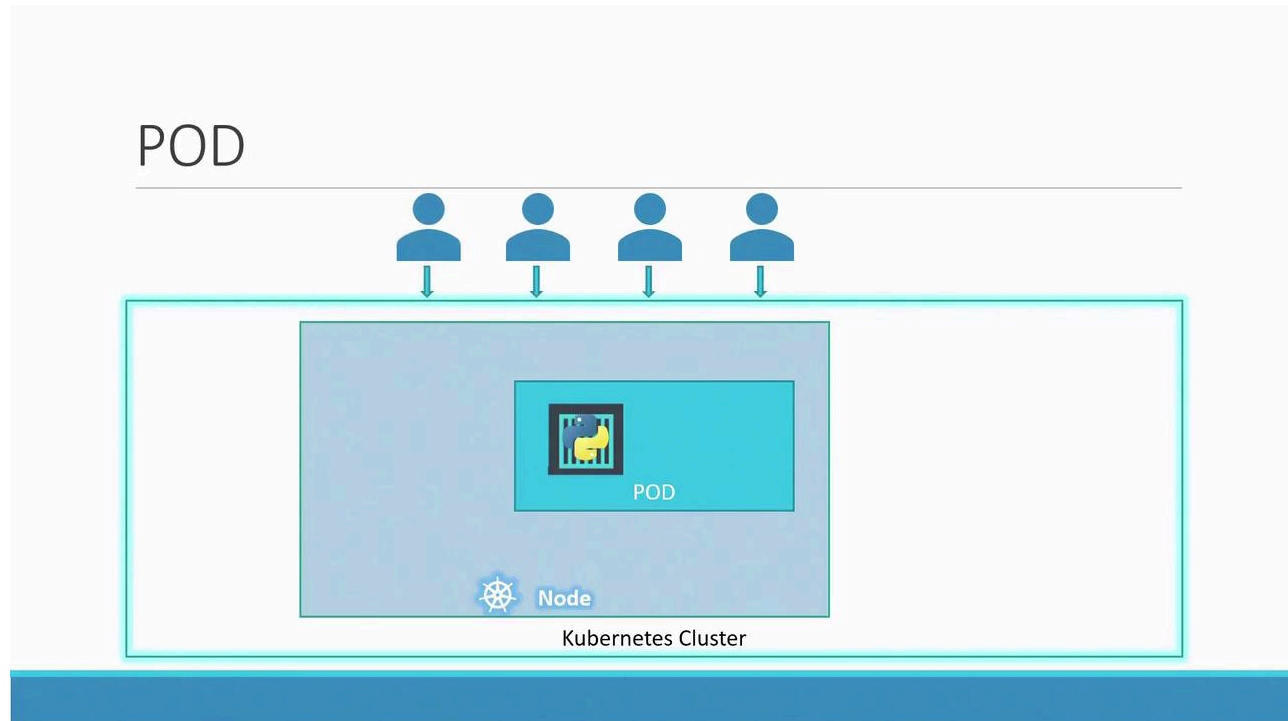
1. An application developed, built into Docker images, and published to a Docker repository (e.g., Docker Hub) for Kubernetes to pull.
2. A fully operational Kubernetes cluster (either single-node or multi-node) up and running.
3. All additional services required by the application are active.

Kubernetes uses pods as the smallest deployable units that encapsulate one or more containers. A pod represents a single instance of your application running on a worker node.

The Role of Pods

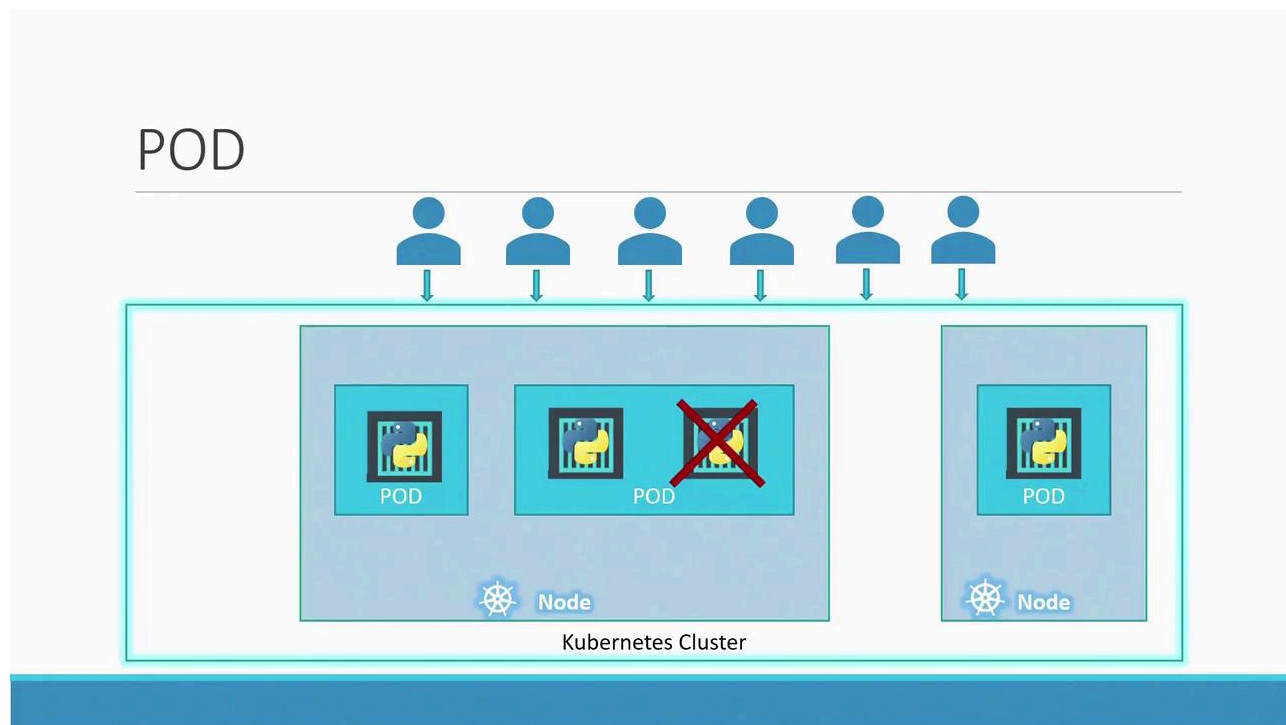
In a basic scenario, you may run a single instance of your application in a Docker container encapsulated within a pod on a single-node cluster. However, as the number of users grows, simply adding more containers inside a single pod is not the solution.

When demand increases and scaling is necessary, you should deploy additional pods, each containing an instance of your application. Consider the following diagram that illustrates a Kubernetes cluster running a Python application inside a pod:



As load increases, more pods are created on the same node or on additional nodes, ensuring efficient distribution of resources and optimal performance.

Another important aspect is that pods typically maintain a one-to-one relationship with containers. To scale up, create new pods; to scale down, delete existing pods. Additional containers are not added to an existing pod for scaling purposes.

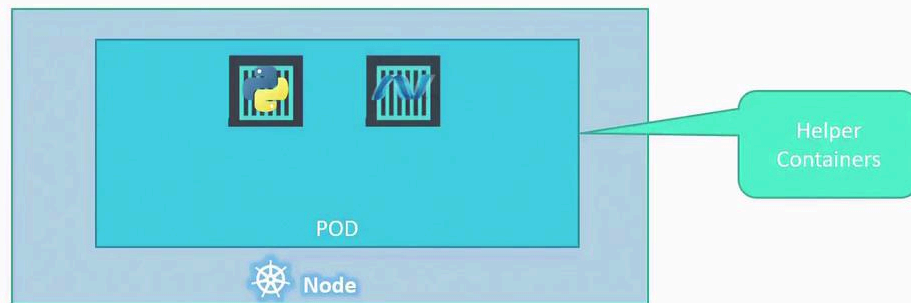


Multi-Container Pods

While most pods run a single container, there are scenarios where running multiple containers together is beneficial. For instance, a helper container may run alongside your primary application container to support functions like data processing or file uploads. In this setup, both containers start and terminate together, sharing the same network namespace and storage volumes.

The diagram below demonstrates a multi-container pod configuration:

Multi-Container PODs



Note

Containers within a pod can communicate with each other using `localhost` since they share the same network context, making inter-container communication seamless.

Benefits of Pods Over Direct Docker Commands

When using Docker directly, you might deploy an application using:

```
docker run python-app
```

Scaling by running multiple container instances might involve:

```
docker run python-app
docker run python-app
```

```
docker run python-app  
docker run python-app
```

If your application evolves to require a helper container, you may find yourself running additional commands like:

```
docker run helper --link app1  
docker run helper --link app2  
docker run helper --link app3  
docker run helper --link app4
```

Managing network connectivity, shared storage, and the lifecycles of interconnected containers manually can be complex and error-prone. Pods encapsulate these responsibilities—ensuring shared networking, storage, and lifecycle management automatically without manual intervention.



Warning

Manual configurations for linking containers in Docker can lead to significant operational overhead. Pods in Kubernetes abstract these complexities, offering streamlined management and deployment.

Deploying a Pod Using kubectl

Deploying a pod in Kubernetes is straightforward with the kubectl command-line tool. For example, to create a pod running the nginx Docker image, execute:

```
kubectl run nginx --image nginx
```

This command pulls the nginx image from Docker Hub by default. For private repositories, additional configurations may be applied.

After deployment, verify the pod status with:

```
kubectl get pods
```

Initially, you may see output similar to:

```
C:\Kubernetes>kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-8586cf59-whssr	0/1	ContainerCreating	0	3s

After a short period, the status will update to indicate that the pod is running:

```
C:\Kubernetes>kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-8586cf59-whssr	1/1	Running	0	8s

At this stage, note that the nginx web server is configured for internal access only. To expose it to end users, additional networking and service configurations are required.

Conclusion

Pods are fundamental to Kubernetes, encapsulating containers and simplifying scaling, deployment, and management. Whether running a single container or a multi-container setup, the pod abstraction provides a robust foundation for building



Beta



● Search docs...

⌘K

Kubernetes and Cloud Native Associate - KCNA ✓

Watch video content

Previous

◀ **Docker vs ContainerD**

Next

Demo Pods ▶